Documento de arquitetura

Documento de Arquitetura de Software

Sistema Vault

Versão 1.0

Modelo estruturado com base no template Git enviado
Adaptado a partir do Documento de Visão e do Diagrama de Casos de Uso do Vault

Histórico de Revisão

Data	Versão	Descrição	Autor
29/03/2026	0.1	Criação do documento de visão	Arthur Benevides, Arthur Dias, Guilherme Neves, Éder Machado, Larissa Carlos
07/04/2026	0.2	Consolidação da arquitetura em camadas	Arthur Benevides, Arthur Dias, Guilherme Neves, Éder Machado, Larissa Carlos
20/04/2026	1.0	Adaptação para o modelo Git e inclusão dos diagramas	Arthur Benevides, Arthur Dias, Guilherme Neves, Éder Machado, Larissa Carlos

Sumário

1. Introdução
1.1 Finalidade
1.2 Escopo
1.3 Definições, Acrônimos e Abreviações
1.4 Visão Geral
2. Representação da Arquitetura
2.1 Diagrama de Relações
3. Metas e Restrições de Arquitetura
4. Visão Lógica
4.1 Visão geral: pacotes e camadas
4.2 Componentes principais
4.3 Diagrama de Classes
5. Visão de Implementação
5.1 Diagrama Entidade Relacionamento
5.2 Diagrama Lógico de Dados
6. Tamanho e Desempenho
7. Qualidade
8. Referências

Nome do Sistema: Vault	Documento de Arquitetura	Versão 1.0

1. Introdução

1.1 Finalidade

Este documento tem como objetivo apresentar uma visão geral da arquitetura de software do sistema Vault e registrar as principais decisões arquiteturais que sustentam o desenvolvimento da solução desktop para busca, download, organização e execução de jogos via BitTorrent.

1.2 Escopo

O Vault é uma aplicação desktop concebida para centralizar a busca em múltiplos indexadores, automatizar downloads via magnet link ou arquivo torrent, enriquecer o catálogo com metadados externos e permitir o gerenciamento da biblioteca local do usuário. O escopo arquitetural contempla a interface desktop, a camada de backend local, a persistência de dados e as integrações com APIs de metadados e indexadores. Não contempla hospedagem de conteúdo, operação da rede P2P ou distribuição comercial de jogos.

1.3 Definições, Acrônimos e Abreviações

Sigla / Termo	Definição
P2P	Comunicação peer-to-peer utilizada pelo protocolo BitTorrent.
IPC	Comunicação entre processos usada na integração entre Electron/Next.js e o backend .NET.
Indexer	Fonte de busca de torrents integrada ao sistema.
Metadata Provider	Serviço externo usado para obter capas, descrições e requisitos, como IGDB e RAWG.

1.4 Visão Geral

O documento está organizado em oito partes: introdução, representação da arquitetura, metas e restrições, visão lógica, visão de implementação, considerações de tamanho e desempenho, atributos de qualidade e referências. A estrutura segue a lógica do modelo Git enviado, mas foi preenchida com o conteúdo específico do sistema Vault.

2. Representação da Arquitetura

O Vault adota uma arquitetura em camadas para uma aplicação desktop moderna. A interface é construída com Electron e Next.js, enquanto a lógica de negócio e o motor de downloads são centralizados em um backend .NET. O sistema integra indexadores de torrents, provedores de metadados externos, banco de dados MySQL e o sistema de arquivos local da máquina do usuário.

Essa organização permite separar responsabilidades, reduzir acoplamento entre interface e regras de negócio, e facilitar evolução incremental do produto.

2.1 Diagrama de Relações

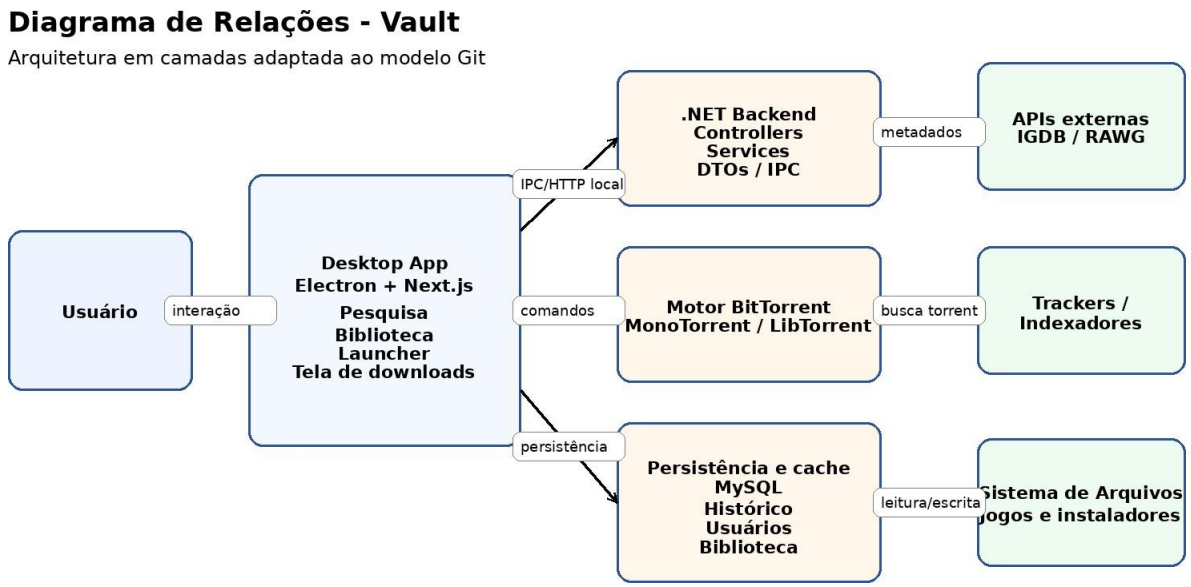

Figura 1 - Diagrama de relações do Vault.

3. Metas e Restrições de Arquitetura

Restrição / Meta	Aplicação no Vault
Linguagem / Stack	Frontend com Next.js/Electron e backend em .NET Core.
Plataforma	Aplicação desktop para Windows 10 ou superior, com expansão futura para Linux.
Persistência	MySQL para usuários, biblioteca, histórico e

	preferências locais.
Segurança	Login obrigatório, armazenamento seguro de credenciais e uso de indexadores conhecidos.
Integração externa	Dependência de APIs de metadados como IGDB/RAWG e de indexadores disponíveis.
Restrições legais	Projeto acadêmico/portfólio, sem comercialização e sem distribuição de conteúdo protegido.

4. Visão Lógica

A visão lógica detalha a divisão do sistema em camadas e componentes, descrevendo como a interface, a lógica de negócio, as integrações e a persistência cooperam para entregar as funcionalidades do produto.

4.1 Visão geral: pacotes e camadas

- Camada de apresentação: Electron Shell, páginas Next.js, componentes de interface, hooks e controladores de formulário.
- Camada de aplicação: autenticação, busca, gerenciamento da biblioteca, execução do launcher e monitoramento de downloads.
- Camada de integração: comunicação IPC, adaptadores para trackers, provedores de metadados e sistema de arquivos.
- Camada de dados: repositórios, entidades, DTOs, histórico de download, usuários e catálogo local.

4.2 Componentes principais

Os componentes centrais identificados a partir do documento de visão e dos casos de uso são:

- User: autenticação, preferências e perfil de acesso.
- Game: catálogo lógico do jogo com capa, descrição, requisitos e status na biblioteca.
- DownloadTask: acompanhamento do download, progresso, peers, seeds e velocidade.
- LibraryEntry: vínculo do usuário com o jogo instalado, concluído ou favoritado.
- Indexer: configuração das fontes de busca utilizadas pelo sistema.
- MetadataProvider: integração com serviços externos para enriquecimento do catálogo.
- SupportTicket: representação simplificada do processo de suporte ao usuário.

4.3 Diagrama de Classes

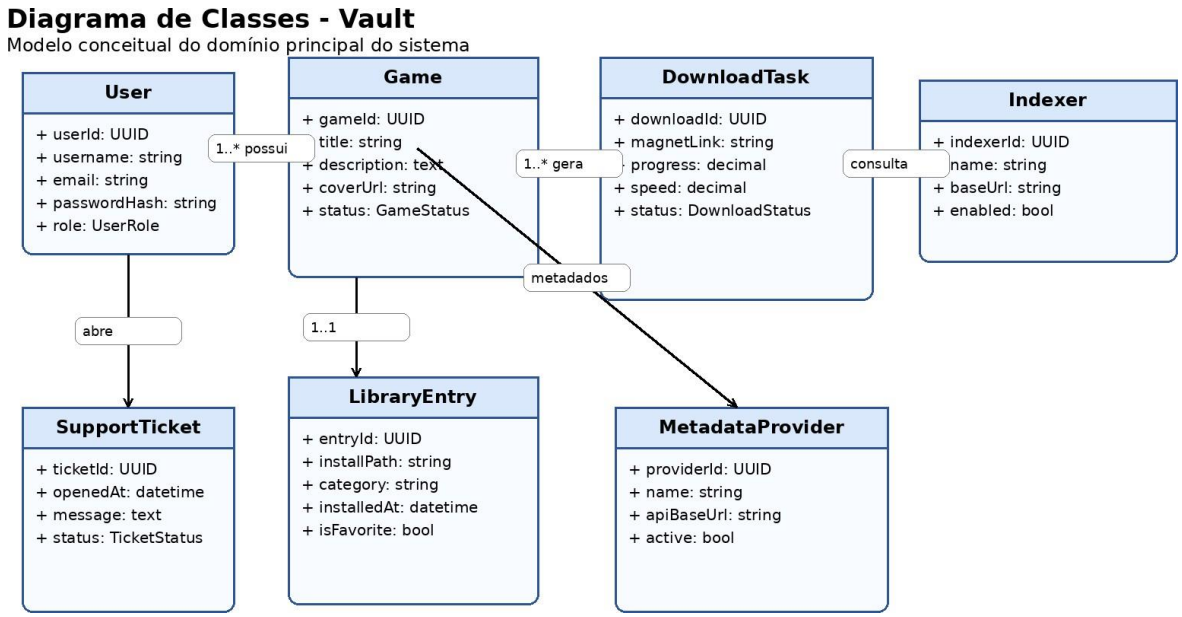

Figura 2 - Diagrama de classes conceitual do Vault.

5. Visão de Implementação

Na implementação, o Vault mantém separação entre frontend, serviços de backend, integrações externas e persistência local. A estrutura sugerida favorece manutenção e crescimento progressivo do código, mantendo o launcher e o motor de torrent desacoplados da interface visual.

5.1 Diagrama Entidade Relacionamento

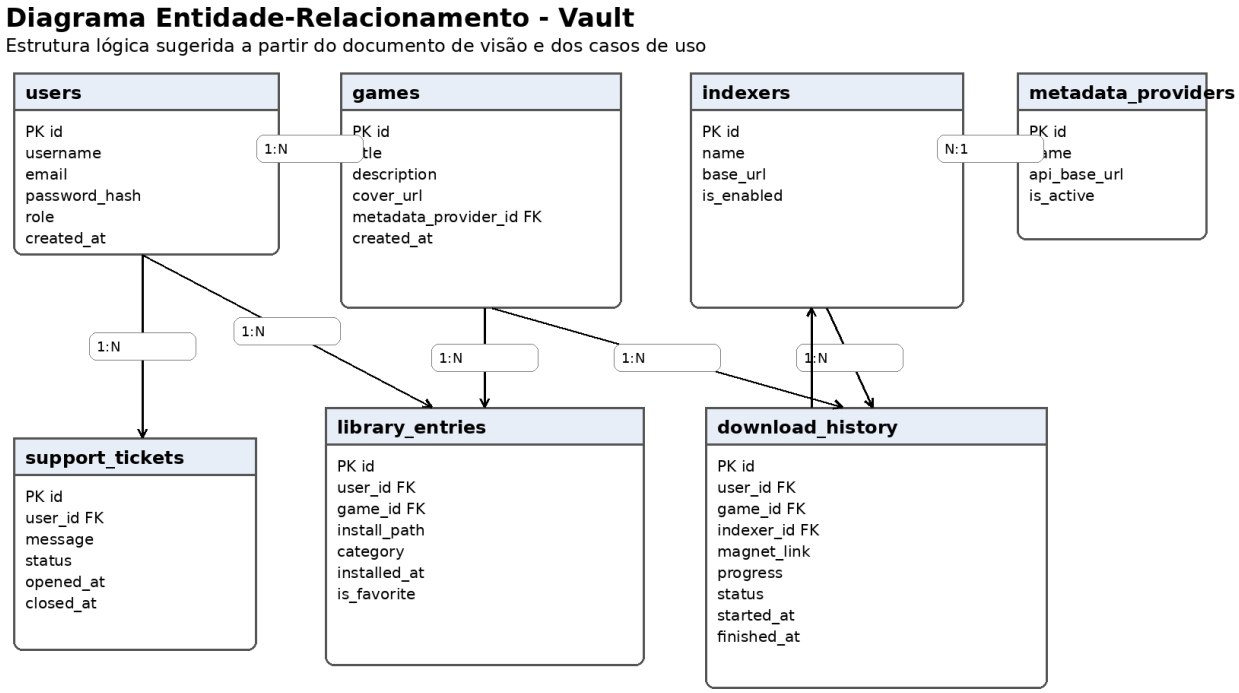

Figura 3 - Diagrama entidade-relacionamento proposto para o Vault.

5.2 Diagrama Lógico de Dados

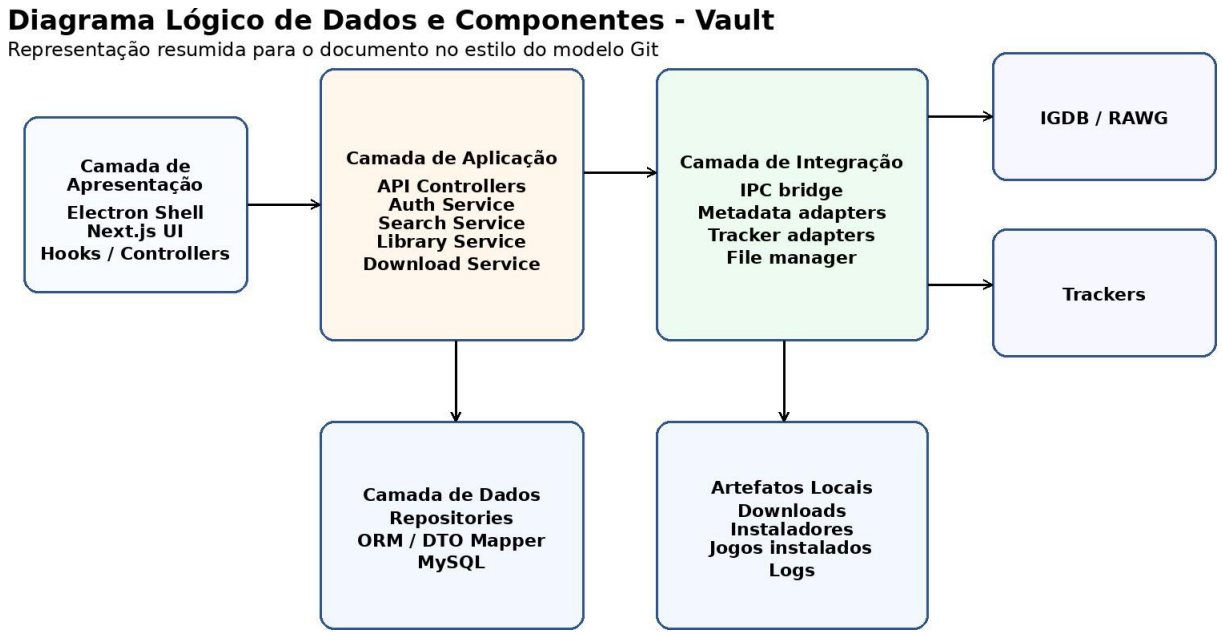

Figura 4 - Diagrama lógico de dados e componentes.

6. Tamanho e Desempenho

O sistema deve operar adequadamente em máquinas com no mínimo 4 GB de RAM, armazenamento suficiente para a biblioteca do usuário e conexão ativa com a internet para busca e download. O backend local deve suportar downloads simultâneos, atualização fluida da interface e consulta rápida da biblioteca instalada. O desempenho percebido depende diretamente da largura de banda disponível, da disponibilidade dos indexadores e das APIs externas, bem como da velocidade de leitura e escrita em disco.

7. Qualidade

- Modificabilidade: separação em camadas facilita troca de componentes e evolução incremental.
- Usabilidade: interface inspirada em launchers modernos, com navegação visual e foco em simplicidade operacional.
- Confiabilidade: monitoramento do download, histórico persistido e verificação de integridade dos arquivos.
- Segurança: autenticação, controle de acesso, criptografia de credenciais e seleção de fontes confiáveis.
- Manutenibilidade: organização por serviços, DTOs, entidades e adaptadores de integração.

- Escalabilidade funcional: novas integrações de indexadores e provedores podem ser incluídas com menor impacto.

8. Referências

- Documento de Visão - Vault, versão 1.0.
- Diagrama de Casos de Uso - Vault.
- Modelo Git de Documento de Arquitetura de Software enviado pelo usuário.